

Lecture 15:

C++ basics

CEE 690

Example 1: Hello world

```
#include <iostream>

int main() {

    std::cout << "Hello World" << std::endl;

    return 0;

}
```

```
(cee690) nc153 $ g++ -o hello_world.exe hello_world.cpp
(cee690) nc153 $ ./hello_world.exe
Hello World
```

Preprocessor directives (#)

```
#include <iostream>
```

- The preprocessor directive tells it to include the entire content of a specified file at that point before compilation
- How is this similar/dissimilar to “import” in Python?

main()

```
int main()
```

- Contrary to Python, you can't have “floating” code. Every instruction must live inside a function.
- The “int” tells the computer what kind of product this function will return.

Return 0

```
return 0;
```

- What value is returned is used to tell us how things went.
- Generally 0 means that main() was executed without any hiccups.

Semicolon (;)

```
return 0;
```

- In C++ “;” is a statement terminator
- C++ does not care about white spaces
- Using the semicolon is critical to know when a command ends and thus another begins

Print statements

```
std::cout << "Hello World" << std::endl;
```

- “std” is the namespace (see it as the library)
- “cout” is the character output stream (see it as the tool that you are using from std)
- “<<” insertion operator (defining what is getting printed)
- “std::endl” -> end of line/flush

iostream - Where is it?

```
#include <iostream>
```

```
(cee690) nc153 $ tail -n 20 /usr/include/c++/11/iostream
extern istream cin;          /// Linked to standard input
extern ostream cout;        /// Linked to standard output
extern ostream cerr;        /// Linked to standard error (unbuffered)
extern ostream clog;        /// Linked to standard error (buffered)

#ifdef _GLIBCXX_USE_WCHAR_T
extern wistream wcin;        /// Linked to standard input
extern wostream wcout;      /// Linked to standard output
extern wostream wcerr;      /// Linked to standard error (unbuffered)
extern wostream wclog;      /// Linked to standard error (buffered)
#endif
///@}

// For construction of filebuffers for cout, cin, cerr, clog et. al.
static ios_base::Init ioinit;
```

When you compile the code, it just includes this piece of code and related code inside your code

The standard library (std)

```
[(cee690) nc153 $ ls
algorithm    complex      exception    memory_resource  stop_token
any          complex.h    execution    mutex            streambuf
array        concepts     experimental new            string
atomic       condition_variable ext          numbers        string_view
backward    coroutine   fenv.h      numeric         syncstream
barrier      csetjmp      filesystem  optional        system_error
bit          csignal      forward_list ostream         tgmath.h
bits       cstdalign    fstream     parallel      thread
bitset       cstdarg      functional  pstl          tr1
cassert      cstdbool     future      queue           tr2
ccomplex     cstddef     initializer_list random          tuple
cctype       cstdint      iomanip     ranges          typeindex
cerrno       cstdio       ios         ratio           typeinfo
cfenv        cstdlib      iosfwd      regex          type_traits
cfloat       cstring      iostream   scoped_allocator unordered_map
charconv     ctgmath      istream     semaphore       unordered_set
chrono       ctime        iterator    set            utility
cinttypes    cuchar       latch       shared_mutex   valarray
ciso646      cwchar       limits      source_location variant
climits      cwctype      list        span           vector
clocale      cxxabi.h     locale      sstream        version
cmath        debug       map         stack          x86_64-redhat-linux
codecvt      decimal    math.h      stdexcept
compare      deque        memory      stdlib.h
```


Example 2: Gravity

```
#include <iostream>

int main() {
    // 1. Variable Declarations (The "Boxes" in memory)
    double y0 = 100.0;    // Initial height in meters
    double v0 = 5.0;      // Initial velocity in m/s
    double g = 9.81;      // Acceleration due to gravity
    double y;             // We will calculate this in the loop

    // 2. The Header Print
    std::cout << "Time(s) | Position(m)" << std::endl;
    std::cout << "-----" << std::endl;

    // 3. The Calculation Loop (0 to 5 seconds)
    for (int t = 0; t <= 5; t++) {

        // C++ Math: Note that we use 0.5 instead of 1/2
        // (In C++, 1/2 would be 0 because it's integer division!)
        y = y0 + (v0 * t) - (0.5 * g * t * t);

        // 4. Output the result
        std::cout << t << "      | " << y << std::endl;
    }

    return 0;
}
```

```
(cee690) nc153 $ g++ -o gravity.exe gravity.cpp
(cee690) nc153 $ ./gravity.exe
Time(s) | Position(m)
-----
0        | 100
1        | 100.095
2        | 90.38
3        | 70.855
4        | 41.52
5        | 2.375
```

Static typing

```
double y0 = 100.0;    // Initial height in meters
```

- In Python we just declare a variable and Python assigns according to what we put there
- In C++ you have to define the type (double, float, int...)
- If you don't define the type it will fail.

```
double v0 = 5.0;    // Initial velocity in m/s
g = 9.81;           // Acceleration due to gravity
double y;           // We will calculate this in the loop
```

```
(cee690) nc153 $ g++ -o gravity.exe gravity.cpp
gravity.cpp: In function 'int main()':
gravity.cpp:8:5: error: 'g' was not declared in this scope
      8 |     g = 9.81;    // Acceleration due to gravity
        |     ^
```

Commenting

```
// 2. The Header Print  
std::cout << "Time(s) | Position(m)" << std::endl;  
std::cout << "-----" << std::endl;
```

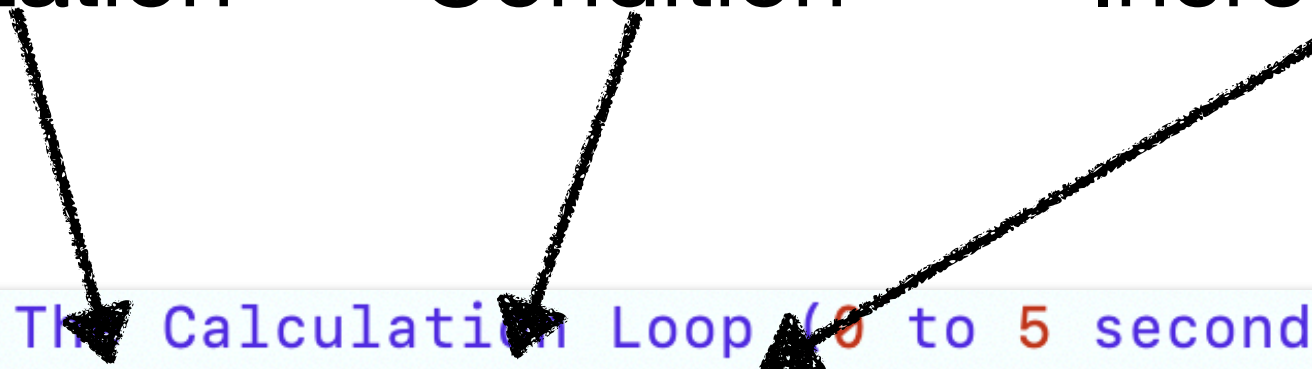
- `//` (think `#` in Python)
- `/* ... */` (block comments)
- `///` or `/**` (for professional documentation)
- In Python we used Sphinx, in C++ we will use Doyxgen

For loops

Initialization

Condition

Increment



```
// 3. The Calculation Loop (0 to 5 seconds)
for (int t = 0; t <= 5; t++) {

    // C++ Math: Note that we use 0.5 instead of 1/2
    // (In C++, 1/2 would be 0 because it's integer division!)
    y = y0 + (v0 * t) - (0.5 * g * t * t);

    // 4. Output the result
    std::cout << t << "          | " << y << std::endl;
}
```

- The scope of `int t` is only within the for loop

Example 3: Spatial mean

```
#include <iostream>

int main() {
    // 1. Initialize a 3x3 grid for simplicity
    double grid[3][3] = {
        {10.2, 11.5, 9.8},
        {12.1, 10.0, 11.2},
        {9.5, 10.8, 11.1}
    };

    double sum = 0.0;
    int rows = 3;
    int cols = 3;

    // 2. Nested Loops: Row-major traversal
    for (int i = 0; i < rows; i++) {           // Outer loop (Rows)
        for (int j = 0; j < cols; j++) {       // Inner loop (Columns)
            sum += grid[i][j];                 // Accumulate the values
        }
    }

    // 3. Calculate Mean
    // Note: total elements is rows * cols
    double mean = sum / (rows * cols);

    std::cout << "The spatial mean of the grid is: " << mean << std::endl;

    return 0;
}
```

```
(cee690) nc153 $ ./spatial_mean.exe
The spatial mean of the grid is: 10.6889
```


Example 4: Spatial stats

```
// --- Main Execution ---
int main(int argc, char* argv[]) {
    json config = get_args(argc, argv);

    SpatialAnalyzer analyzer(config);
    analyzer.run_analysis();
    analyzer.visualize();
    analyzer.save_netcdf();

    std::cout << "Processing complete." << std::endl;
    return 0;
}
```

Compiling (Linking is where this gets “fun”!)

```
(cee690) nc153 $ g++ spatial_stats.cpp -o spatial_stats.exe -I/hpc/home/nc153/miniconda3/envs/cee690/include -I/hpc/home/nc153/miniconda3/envs/cee690/include/eigen3 -L/hpc/home/nc153/miniconda3/envs/cee690/lib -Wl,-rpath,/hpc/home/nc153/miniconda3/envs/cee690/lib -lnetcdf_c++4 -lnetcdf
```

Running

```
(cee690) nc153 $ ./spatial_stats.exe --JSON_FILE example.json
Configuration loaded from example.json
Loading dataset: /hpc/home/nc153/spatialstats/data/era_interim_annual_197901_201512_upscaled_subset.nc
Computing the statistics...
Visualizing the data...
Plot saved to output_plot.png
Saving statistics to output_temporal_stats.nc
Processing complete.
```


Example 4: Spatial stats

Number of input arguments

Array of strings (e.g., argv[0])



```
// --- Main Execution ---  
int main(int argc, char* argv[]) {  
    json config = get_args(argc, argv);  
  
    SpatialAnalyzer analyzer(config);  
    analyzer.run_analysis();  
    analyzer.visualize();  
    analyzer.save_netcdf();  
  
    std::cout << "Processing complete." << std::endl;  
    return 0;  
}
```